

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии

Дисциплина «Введение в программирование и алгоритмы»

Отчет по лабораторной работе №3
«Ветвления. Логические операторы»

Выполнил студент группы
ПИЖ-б-о-26-1(2)
Киркачев М.Е.
Проверил:
Преподаватель по дисциплине
Николаев Е. И.

г. Ставрополь, 2026 г.

Цель работы: Научиться использовать условный оператор if-else для реализации ветвлений в программах, применять логические операторы (&&, ||, !) для построения сложных условий, реализовывать защиту от некорректного ввода (деление на ноль, проверка диапазонов), создавать простое консольное меню с выбором действия.

ХОД РАБОТЫ

https://github.com/Max2311008/my-projects/tree/main/work%20in%20C++/lub_3

Вариант №9

Задание 1(базовый уровень)

Код:

```
#define NOMINMAX // запретить Windows.h определять макросы min и max
#include <iostream> // Подключаем заголовочный файл
#include <Windows.h>
#include <cmath>
#include <iomanip> // библиотека для fixed и setprecision()
int main() { // объявляем главную функцию
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    // 3,8,9 строки обрабатывают кириллицу
    double number; // создание переменной с плавающей точкой
    while (true) {
        std::cout << "Введите число: ";
        if (std::cin >> number) {
            break;
        }
        // ввод числа с повтором ввода данных при ошибке
        std::cout << "ОШИБКА! Введите число заново.\n"; // ввод текста на экран
        std::cin.clear(); // сброс флага ошибки
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        // очистка буфера от мусора
    }
    if (number < 0) {
        std::cout << "\n === Ответ(модульный) до округления === " << fabs(number) << std::endl;
        std::cout << "\n === ОТВЕТ === " << std::fixed << std::setprecision(2) << fabs(number) << std::endl;
    }
    else {
        std::cout << "\n === Ответ до округления === " << number << std::endl;
        std::cout << "\n === ОТВЕТ === " << std::fixed << std::setprecision(2) << number << std::endl;
    }
    /*Условие при котором если число отрицательное,
    то выводим модуль этого числа или само число */
    return 0; // Вывод результата
}
```

Результат:

```
Введите число: 424.423

=== Ответ до округления === 424.423

=== ОТВЕТ === 424.42
```

```
Введите число: -424.4222

=== Ответ(модульный) до округления === 424.422

=== ОТВЕТ === 424.42
```

Задание 2 (средний уровень)

Код:

```
// очистка буфера от мусора
}
while (true) {
    std::cout << "Введите число 2: ";
    if (std::cin >> number2) {
        break;
    }
    // ввод числа с повтором ввода данных при ошибке
    std::cout << "ОШИБКА! Введите число заново.\n"; // ввод текста на экран
    std::cin.clear(); // сброс флага ошибки
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    // очистка буфера от мусора
}
while (true) {
    std::cout << "Введите число 3: ";
    if (std::cin >> number3) {
        break;
    }
    // ввод числа с повтором ввода данных при ошибке
    std::cout << "ОШИБКА! Введите число заново.\n"; // ввод текста на экран
    std::cin.clear(); // сброс флага ошибки
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    // очистка буфера от мусора
}
if (number1 <= number2 && number2 <= number3) {
    std::cout << "\n === Ответ === | " << number1 << " | " << number2 << " | " << number3;
}
else if (number2 <= number1 && number1 <= number3) {
    std::cout << "\n === Ответ === | " << number2 << " | " << number1 << " | " << number3;
}
else if (number3 <= number2 && number2 <= number1) {
    std::cout << "\n === Ответ === | " << number3 << " | " << number2 << " | " << number1;
}
else if (number3 <= number1 && number1 <= number2) {
    std::cout << "\n === Ответ === | " << number3 << " | " << number1 << " | " << number2;
}
else if (number1 <= number3 && number3 <= number2) {
    std::cout << "\n === Ответ === | " << number1 << " | " << number3 << " | " << number2;
}
else if (number2 <= number3 && number3 <= number1) {
    std::cout << "\n === Ответ === | " << number2 << " | " << number3 << " | " << number1;
}
// Условия для записи трёх чисел в возрастающем порядке

return 0; // Вывод результата
}
```

Результат:

```
Введите число 1: 45
Введите число 2: 4353
Введите число 3: 942

=== Ответ === | 45 | 942 | 4353
```

Задание 3 (повышенный уровень)

Код:

```
double summa;
std::string status;
while (true) {
    std::cout << "Введите число: ";
    if (std::cin >> summa) {
        break;
    }
    // ввод числа с повтором ввода данных при ошибке
    std::cout << "ОШИБКА! Введите число заново.\n"; // ввод текста на экран
    std::cin.clear(); // сброс флага ошибки
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    // очистка буфера от мусора
}

std::cout << "Введите статус клиента(обычный| VIP | сотрудник): ";
std::cin >> status; // Ввод с клавиатуры
double discout = 0; // Создание переменной скидки
if (status == "обычный") {
    discout = 0;
}
else if (status == "VIP") {
    discout = 10;
}
else if (status == "сотрудник") {
    discout = 20;
}
else {
    std::cout << "\nСтатус не найден, скидки не будет\n";
}
if (summa > 5000) {
    discout += 5;
}

// Условия для обозначения скидок для статуса
double otvet = summa - ((summa * discout) / 100);
std::cout << "\n=== Результат ===\n";
std::cout << "Сумма покупки: " << std::fixed << std::setprecision(4) << summa << " руб.\n";
std::cout << "Скидка на покупку: " << discout << " %\n";
std::cout << "Итоговая сумма: " << std::fixed << std::setprecision(4) << otvet << " руб.\n";
// Вывод на экран итоговую сумму с округлением после 4 знака
return 0; // Вывод результата
}
```

Результат:

```
Введите число: 50000
Введите статус клиента(обычный| VIP | сотрудник): VIP

=== Результат ===
Сумма покупки: 50000.0000 руб.
Скидка на покупку: 15.0000 %
Итоговая сумма: 42500.0000 руб.
```

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. = - оператор присваивания. Он присваивает значение правого операнда левому
== - оператор сравнения. Проверяет, равны ли значения
2. Произойдёт присваивание 5 переменной x, результатом будет 5
3. `if (x >= 10 && x <= 20) {`
4. `&&` - оба операнда истинны
`||` - хотя бы один операнд истинен
5. Ленивые вычисления – это поведение логических операторов `&&` и `||`, при котором правая часть выражения не вычисляется, если результат уже ясен. Защита — это когда мы ставим проверку безопасности слева, а опасное действие справа. Тогда при плохих данных опасное действие просто не выполнится.
6. Нельзя сравнивать операнд `double` через `==` из-за погрешности чисел с плавающей точкой. Правильный способ: `if (std::fabs(a - b) < 1e-9 {..}`
7. `Std::cin.fail()` – проверяет, на наличие ошибки в вводе
`Std::cin.clear()` – сбрасывает флаги ошибок, чтобы `cin` снова работал
8. Чтобы избежать ошибок при добавлении кода и явно задать границы блока
9. `X = 5` – это истинно, значит выведет `True`
10. Тернарный оператор – это сокращённый `if-else`
`int max = (a > b) ? a : b;`
11. Через `switch` или `if / else if`
12. Потому что `else if` проверяется только если предыдущее условие ложно, а отдельные `if` проверяют всё, что может привести к лишним действиям и возможным ошибкам
13. В, так как `if(0)` – ложь, а В находится не в `if`
- 14.

```

#define NOMINMAX
#include <iostream>
#include <limits>

int main() {
    int number;
    std::cout << "Введите число: ";

    if (std::cin >> number) {
        std::cout << "Вы ввели число: " << number << "\n";
    }
    else {
        std::cout << "Ошибка: введено не число!\n";

        std::cin.clear(); |
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }
    return 0;
}

```

15. Вложенные if – это if внутри if, используются для последовательной проверки нескольких условий. Else всегда связывается с ближайшим предыдущим if, у которого ещё нет else, независимо от отступов и переносов строк. Лучше всегда использовать {} даже для одной строки. Для более лучшей читаемости, лучше объединить условия через &&.

Вывод: Благодаря этой лабораторной работе я Научился использовать условный оператор if-else для реализации ветвлений в программах, применять логические операторы (&&, ||, !) для построения сложных условий, реализовывать защиту от некорректного ввода (деление на ноль, проверка диапазонов), создавать простое консольное меню с выбором действия.